KEYFRAME AUDIO VIA EXTREMA SAMPLING

Anonymous Authors *

Dept. of Anonymous Studies
Anonymous Affiliation
Anonymous Location
an@nym.ous

ABSTRACT

In this paper we present an alternative 1D signal representation inspired by keyframe animation. Our method reduces uniformly sampled signals down to only their local extrema. We demonstrate content-aware time stretching directly from the sparse format in stereo at 48 kHz on a 480 MHz STM32H7 microcontroller. For each test signal we measure performance at three levels of quality. Compression ratios average 3:1 and 8.7:1 at high and low quality settings, with CPU utilization averaging 6-8% even at extreme time stretch ratios. Results show strong preservation of transients and harmonic content with the primary artifacts being increased saturation and broadband noise reminiscent of the sound of analog tape.

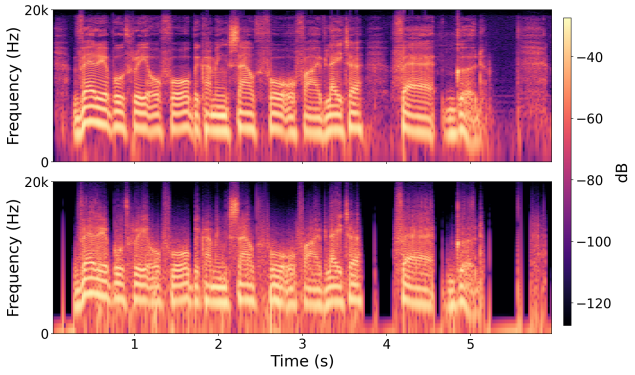

Figure 1: Speech spectrogram before and after 15:1 compression (low quality setting).

1. INTRODUCTION

In uniformly sampled audio, the sample rate is a fixed value that determines the highest frequency the system can represent. For audio, 44.1 kHz is often chosen because its maximum frequency is the limit of human hearing. However, this means a bass note and a cymbal crash require the same number of samples per second despite having fundamentally different information densities. In 3D graphics, a vector-based approach is often used instead, where sparse keyframes represent complex paths. In this paper we apply a similar philosophy to 1D audio signals. By reducing a waveform to only its timestamped local extrema, we obtain a sparse format that naturally adapts the sample rate to the signal's complexity.

A practical consequence of this representation is significant data reduction which directly addresses RAM and bandwidth constraints in embedded audio hardware. More importantly, the sparse structure encodes the signal's information in a way that enables operations like time stretching at virtually no added cost, without requiring FFT or autocorrelation analysis.

Unlike perceptual codecs designed for transparency, our goal is not perfect reconstruction but a structurally faithful, computationally efficient signal description suitable for embedded DSP and creative manipulation.

MP3 [1] provides compression based on the assumption that the harmonic content of many "natural" signals is sparse in the frequency domain. Our method makes the same assumption but from a time domain perspective where the relative prominence of an extremum determines if it's considered signal or noise.

Empirical mode decomposition [2] is an alternative to the Fourier transform based upon an iterative sifting process. Our method is inspired by their use of derivatives to identify extrema. We improve upon their extrema detection by using spline derivatives instead of finite differences.

Level-crossing sampling [3] methods replace uniform samples with asynchronous, timestamped data that's triggered by amplitude threshold crossings. We use the same data structure but trigger on extrema instead of amplitude.

Recent work on topology-preserving deformations of audio signals [4] shows that the topological structure of a signal changes only at local extrema. This provides some mathematical foundation for why sampling extrema alone appears to sufficiently represent signal features in our keyframe approach.

2. ALGORITHM DESCRIPTION AND THEORY

2.1. Overview

Our analysis method begins by taking the first derivative of the uniformly sampled input signal, then with every tick checking if the derivative's polarity has changed. When it does, we know a local extremum lies somewhere between those two points.

Next, we take the absolute difference between the potential keyframe's value and the value of the previously saved keyframe. If the difference doesn't exceed a threshold, the potential keyframe is discarded. If it does, we use reverse linear interpolation to find the subsample index where the derivative equals zero, resulting in a refined estimate of the extremum's location.

Next, we evaluate a cubic B-spline at this subsample index to obtain the final keyframe value, storing it alongside the refined

* Thanks to the anonymous supporters

temporal index as a (index, value) pair. This is the compressed storage format that we reconstruct from at runtime.

To reconstruct a continuous time signal from the sparse buffer we use non-uniform cubic interpolation with the tangent calculations optimized out. For time scale modification we use a traditional overlap-add method but define the error between pointers in the sparse domain, resulting in transient preserving, adaptive grain size.

Algorithm 1 Extrema Analysis

Require: Uniform input buffer $x[n]$, threshold ϵ
Ensure: Sparse keyframe buffer $x[m]$
$d_{\text{prev}} \leftarrow 0, \quad v_{\text{prev}} \leftarrow x[0]$
Save keyframe $(0,\ x[0])$ ▷ anchor
for each sample n **do**
  $d[n] \leftarrow \mathcal{B}'(x,\ n)$ ▷ derivative
  if $\text{sign}(d[n]) \neq \text{sign}(d_{\text{prev}})$ **then**
    if $|x[n] - v_{\text{prev}}| > \epsilon$ **then** ▷ above threshold
      $\alpha \leftarrow \frac{|d[n-1]|}{|d[n-1]|+|d[n]|}$
      $n_m \leftarrow (n-1) + \alpha$ ▷ subsample index
      $v_m \leftarrow \mathcal{B}(x,\ n_m)$ ▷ subsample value
      Save keyframe $(n_m,\ v_m)$
      $v_{\text{prev}} \leftarrow v_m$
    end if
  end if
  $d_{\text{prev}} \leftarrow d[n]$
end for

Algorithm 2 Sparse Reconstruction

Require: Sparse buffer $x[m]$ of $(n_m,\ v_m)$ pairs, output length N
Ensure: Uniform output buffer $y[n]$
$m \leftarrow 0$
for each output sample $n = 0 \ldots N-1$ **do**
  while $n_{m+1} < n$ **do** ▷ advance pointer
    $m \leftarrow m + 1$
  end while
  $t \leftarrow (n - n_m)\ /\ (n_{m+1} - n_m)$
  $y[n] \leftarrow v_m\, h_{00}(t) + v_{m+1}\, h_{10}(t)$ ▷ $T_0 = T_1 = 0$
end for

2.2. Bandlimited Derivative Estimation

Windowed sinc interpolation is a kernel convolution method where a window function is applied to the sinc function, confining its infinite support to a finite interval. This kernel is then convolved with the signal. Cubic Hermite splines are an approximation of a windowed sinc kernel. Cubic basis splines, or B-splines, are an approximation of the window itself. When convolved with the signal they offer C^2 continuity and a strong zero at the Nyquist frequency at the cost of some high frequency rolloff.

In practice, we implement B-splines as 4-tap FIR filters where the coefficients are defined by the data itself, requiring no evaluation of trig functions or lookup table interpolation at runtime. The interpolated value in the interval $t \in [0, 1]$ between control points p_{-1}, p_0, p_1, and p_2 is defined as

$$p(t) = p_{-1}b_{-1}(t) + p_0 b_0(t) + p_1 b_1(t) + p_2 b_2(t) \qquad (1)$$

where the cubic equation for each segment is:

$$b_{-1}(t) = \frac{(1-t)^3}{6}, \qquad b_0(t) = \frac{3t^3 - 6t^2 + 4}{6},$$
$$b_1(t) = \frac{-3t^3 + 3t^2 + 3t + 1}{6}, \quad b_2(t) = \frac{t^3}{6}. \qquad (2)$$

Just as a bandlimited derivative can be obtained by windowing the derivative of the sinc function, we take the derivative of each B-spline segment to obtain a bandlimited derivative kernel:

$$b'_{-1}(t) = \frac{-3(1-t)^2}{6}, \qquad b'_0(t) = \frac{9t^2 - 12t}{6},$$
$$b'_1(t) = \frac{-9t^2 + 6t + 3}{6}, \qquad b'_2(t) = \frac{3t^2}{6}. \qquad (3)$$

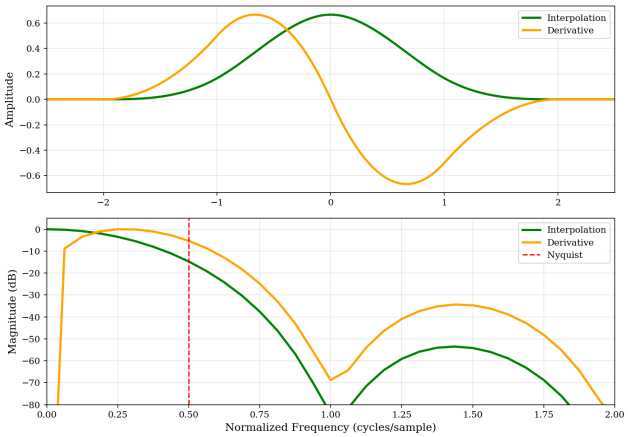

Figure 2: The impulse and frequency responses of the B-spline kernel and its first derivative.

2.3. Difference Thresholding

We start by saving the first sample as a keyframe to act as an anchor. After that, to determine if we should save the current sample as a keyframe or not we consider its absolute difference compared to the last saved keyframe. If the difference in amplitude doesn't exceed a threshold, the keyframe is discarded.

This state-dependent thresholding, where each saved keyframe resets the reference for the next decision, produces a hysteresis-like behavior analogous to the coercivity threshold in magnetic recording media. In the case of digital audio, the signal's amplitude exists on the range -1.0 to 1.0. We found that an amplitude difference threshold of 0.001 provides a high quality sound.

A higher threshold means more compression, at the cost of low amplitude elements of the signal being discarded. High frequencies are preserved even at high thresholds as long as their amplitude is large enough. A lower threshold can only increase the sound quality so much. At a certain point, lowering it just captures the noise floor and runs the risk of increasing file size rather than reducing it.

2.4. Subsample Extrema Location

Once we've determined that an extremum is worth keeping, we can't simply treat the current sample itself as the extremum. Doing

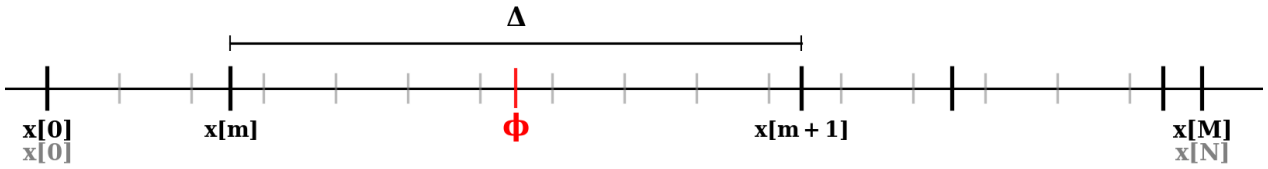

Figure 3: Comparing the uniform time grid of buffer $x[n]$ with its sparse equivalent $x[m]$, where $M < N$ and ϕ represents a pointer's continuous index within the buffer.

so provides a decent approximation for low frequencies but causes significant aliasing as frequency increases.

We utilize reverse linear interpolation to solve for the subsample location where the derivative equals zero. The reduction in aliasing between grid snapping and reverse linear interpolation is significant, but we found diminishing returns when using a secondary linear refinement or a quadratic root finding approach. The algorithm for reverse linear interpolation of the derivative signal $d[n]$ is as follows:

$$\alpha = \frac{|d[n-1]|}{|d[n-1]| + |d[n]|} \tag{4}$$

The resulting subsample index n is then defined as:

$$n_m = (n-1) + \alpha \tag{5}$$

The refined value v_m is obtained by evaluating the B-spline function $\mathcal{B}$ at the refined index n_m:

$$v_m = \mathcal{B}(n_m) \tag{6}$$

2.5. Sparse Buffer Playback

Before we reconstruct the original signal from its sparse representation we must establish a mental model of their relationship. $x[n]$ is a uniform buffer of length N, and $x[m]$ is a sparse buffer of length M, where $N > M$ but both buffers represent the same amount of time. Each keyframe is defined by the pair (n_m, v_m), where n_m is the extremum's subsample index and v_m is its value. Figure 3 visualizes a uniform buffer and its sparse equivalent overlaid on the same timeline.

We use an analog playhead analogy to conceptualize playback, where playing the sparse buffer like tape means incrementing a pointer through its indices. For a particular continuous index, there is a corresponding pair of keyframes needed to produce an output. We call this pair a window. Each w contains:

- The window's sparse index m
- Start and end indices n_m and n_{m+1}
- Duration $\Delta = n_{m+1} - n_m$
- Phase increment $\dot{\phi} = 1/\Delta$

Each pointer consists of:

- A continuous uniform index ϕ
- The appropriate w for that index
- Sparse subsample position $t = (\phi - n_m) \cdot \dot{\phi}$
- Playback speed σ

Finding the correct window for a given ϕ means searching $x[m]$ for the pair where $n_m \leq \phi < n_{m+1}$. A linear scan from a last known valid m keeps the search local, and a binary search bounded around m can be used for random access.

Consider the example of pitched up playback where $\sigma = 1.25$. Each tick, sparse subsample position t is computed from ϕ and the current window and used to interpolate between the two keyframes that make up the window at that continuous index. When $t > 1$, a local search finds the next valid window. At unity speed ($\sigma = 1$), this reproduces the original recording.

With this analogy, we only need to consider the playhead moving along the virtual tape at some speed. All of the sparse to uniform mapping is handled behind the scenes, which allows us to more easily expand on this idea.

2.6. Non-Uniform Cubic Interpolation

Now that we've established the connection between the uniform and sparse domains, it's time to describe how we actually generate interpolated values for a particular ϕ. To do so, we again turn to cubic splines, but this time with a non-uniform spacing between samples. For a given window and t, the interpolated value is defined as:

$$p(t) = v_m h_{00}(t) + T_0 h_{01}(t) + v_{m+1} h_{10}(t) + T_1 h_{11}(t) \tag{7}$$

where $t \in [0, 1]$ is the normalized time between the two keyframes, and the basis functions are

$$\begin{aligned} h_{00}(t) &= 2t^3 - 3t^2 + 1, & h_{01}(t) &= t^3 - 2t^2 + t, \\ h_{10}(t) &= -2t^3 + 3t^2, & h_{11}(t) &= t^3 - t^2. \end{aligned} \tag{8}$$

In a non-uniform cubic Hermite spline, the derivatives, commonly called the tangents in this context, are estimated using finite differences across varying time intervals:

$$T_0 = \frac{1}{2}\left(\frac{v_{m+1} - v_m}{n_{m+1} - n_m} + \frac{v_m - v_{m-1}}{n_m - n_{m-1}}\right) \tag{9}$$

$$T_1 = \frac{1}{2}\left(\frac{v_{m+2} - v_{m+1}}{n_{m+2} - n_{m+1}} + \frac{v_{m+1} - v_m}{n_{m+1} - n_m}\right) \tag{10}$$

The derivative calculations are computationally expensive, requiring two additional keyframes besides m and $m + 1$ and four floating point divisions. However, our analysis already ensured that every keyframe we saved has a derivative of zero.

By setting T_0 and T_1 to zero, the two tangent terms in $p(t)$ vanish, leaving us with the following equation:

$$p(t) = v_m h_{00}(t) + v_{m+1} h_{10}(t) \tag{11}$$

The result depends only on the extremum's value scaled by the cubic equations. The spacing between points is no longer part of the

equation. This means that C^1 continuous non-uniform interpolation is possible using only two keyframes, no division, and less than half the multiplications. With this continuous time representation of the sparse signal we can generate a uniformly sampled output at any desired sample rate, just like traditional ring buffers.

2.7. Time Stretching

Time stretching is an operation that makes pitch and duration independent of one another. Algorithm 3 outlines how we accomplish this. We use a modified version of the classic overlap-add techniques [5, 6] where we keep track of three pointers:

- A **reference pointer** ϕ_{ref} that advances at the time rate τ and represents where playback *should* be.
- A **playhead** ϕ_{play} that produces audio and advances at the pitch rate σ.
- A **temporary pointer** ϕ_{temp} that produces audio at pitch rate σ only while splicing.

Algorithm 3 Sparse-Domain Time Stretching

Require: Sparse buffer $x[m]$, time rate τ, pitch rate σ, threshold K
Ensure: Output sample y per tick

$\phi_{\text{ref}}, \phi_{\text{play}} \leftarrow 0$, splicing $\leftarrow$ false
for each output tick **do**
  Update w_{ref}, w_{play} if $t > 1$ ▷ window search
  $d \leftarrow |w_{\text{play}}.m - w_{\text{ref}}.m|$ ▷ measure distance
  if ¬ splicing **and** $d > K$ **then** ▷ initialize splice
    $\phi_{\text{temp}} \leftarrow \phi_{\text{ref}}$, $w_{\text{temp}} \leftarrow w_{\text{ref}}$
    $L \leftarrow \sum_{i=0}^{K-1} \Delta_{w_{\text{ref}}.m+i}$
    $t_{\text{temp}} \leftarrow 0$, $\dot{t}_{\text{temp}} \leftarrow 1/L$
    splicing $\leftarrow$ true
  end if
  if splicing **and** $t_{\text{temp}} \leq 1$ **then** ▷ conduct splice
    $y_{\text{play}} \leftarrow \text{Interp}(x[m], \phi_{\text{play}})$
    $y_{\text{temp}} \leftarrow \text{Interp}(x[m], \phi_{\text{temp}})$
    $y \leftarrow y_{\text{temp}} \cdot t_{\text{temp}} + y_{\text{play}} \cdot (1 - t_{\text{temp}})$
    $\phi_{\text{play}} \mathrel{+}= \sigma$, $\phi_{\text{temp}} \mathrel{+}= \sigma$
    $t_{\text{temp}} \mathrel{+}= \dot{t}_{\text{temp}} \cdot \sigma$
    $\phi_{\text{ref}} \mathrel{+}= \tau$
    return y
  else if splicing **and** $t_{\text{temp}} > 1$ **then** ▷ finish splice
    $\phi_{\text{play}} \leftarrow \phi_{\text{temp}}$, $w_{\text{play}} \leftarrow w_{\text{temp}}$
    splicing $\leftarrow$ false
  end if
  $y \leftarrow \text{Interp}(x[m], \phi_{\text{play}})$ ▷ normal playback
  $\phi_{\text{play}} \mathrel{+}= \sigma$, $\phi_{\text{ref}} \mathrel{+}= \tau$
  return y
end for

When $\tau \neq \sigma$, error accumulates between the reference and playhead positions. We measure this distance d as the absolute difference between their sparse indices $w.m$. When d exceeds a threshold K (measured in keyframes), a splice is initiated. A temporary pointer spawns at the reference position with the same pitch rate as the playhead. It crossfades in while the playhead fades out. The crossfade duration is the sum of the next K window durations ahead of the reference:

$$L = \sum_{i=0}^{K-1} \Delta_{w_{\text{ref}}.m+i} \tag{12}$$

The crossfade phase t_{temp} increments by $\dot{t}_{\text{temp}} \cdot \sigma$ per tick, so the crossfade completes in L/σ output samples. Once t_{temp} exceeds one, the playhead adopts the temporary pointer's position and window, and normal playback resumes until the distance is exceeded again. During a transient, extrema are densely spaced and L is short. During a bass tone, extrema are far apart and L is long. Figure 5 illustrates this adaptive behavior. At 0.47 seconds where the signal transitions from sparse to dense, the number of keyframes between pointers suddenly becomes large and triggers a splice, preserving transients automatically. Pitch shifting is a reciprocal operation of time stretching as we've described it here. To obtain this behavior, one simply needs to change the pitch rate σ.

3. RESULTS AND DISCUSSION

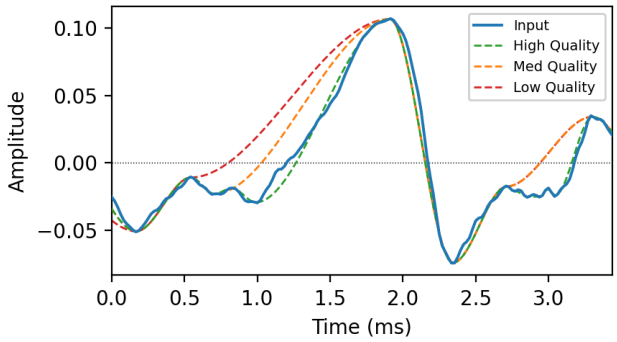

Figure 4: Time domain comparison of three levels of compression on a short musical excerpt.

3.1. Test Methodology

To quantify the performance of this algorithm we employ the Electrosmith Daisy Patch SM eurorack module, with its 480 MHz STM32H7 processor, 24-bit 48 kHz audio codec, and 64 MB SDRAM. We've curated 38 music and speech examples, each six seconds long. We played the samples from the Elektron Digitakt II into the Patch SM with the reconstructed results saved to SD card in wav format. We then use Python to analyze the results and create the figures shown here.

3.2. Compression Performance

For each of the sources, we analyze with low, medium, and high quality thresholds of 0.1, 0.01, and 0.001 respectively. We compare the reconstructed outputs with $\sigma = 1$ against differential PCM (IMA-ADPCM) and a version of the signal downsampled to 7 kHz. Each keyframe is stored as two 32-bit floats (index and value), whereas the input is a single 32-bit float (see further work for discussion of 16 bit quantization), so the compression ratio is defined as:

$$R = \frac{N}{2M} \tag{13}$$

Because this distortion is phase-coherent and signal-dependent,

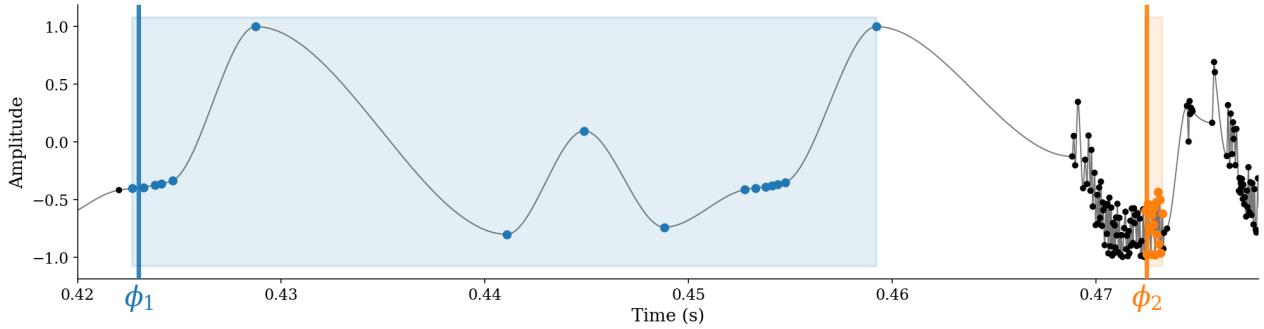

Figure 5: Demonstrating the adaptive nature of grain size on a bass note followed by a snare hit. The 16 keyframes ahead of pointer ϕ_1 have a much longer duration when summed than the 16 keyframes ahead of pointer ϕ_2.

Table 1: Compression ratio and SNR averaged over 38 trials per type

Method	Ratio	SNR (dB)
High quality	3:1	16.69
Med quality	3.9:1	16.0
Low quality	8.7:1	9.1
IMA-ADPCM	4.00:1	34.54
7 kHz DS	6.86:1	14.17

SNR should be interpreted as an aggregate error rather than a perceptual transparency indicator.

3.3. Aliasing and Harmonic Distortion

To measure aliasing of the system as a whole, we compressed a sine wave frequency sweep from 20 kHz to 20 Hz locally on the Patch SM. The reconstruction's spectrogram in Figure 6 shows the harmonics caused by the cubic spline's inability to reproduce a perfect sine wave.

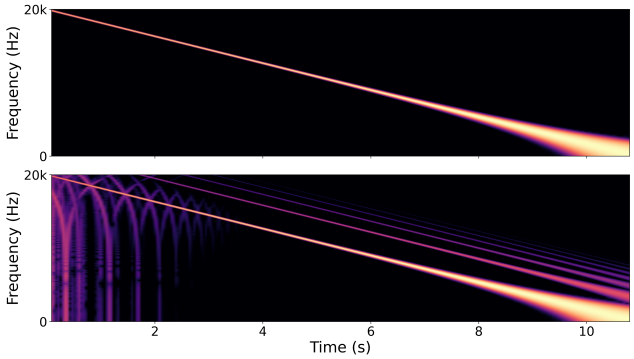

Figure 6: Sine sweep before and after compression. Frequency axis is log scaled.

To quantify the spline's total harmonic distortion, we compressed a 1 kHz sine wave and used the Goertzel algorithm to measure the presence of even and odd harmonics. The total THD is -38.1 dB for odd and -85.8 dB for even. For reference, a tanh saturator at unity gain has a THD of -25 dB for odd harmonics and none for even.

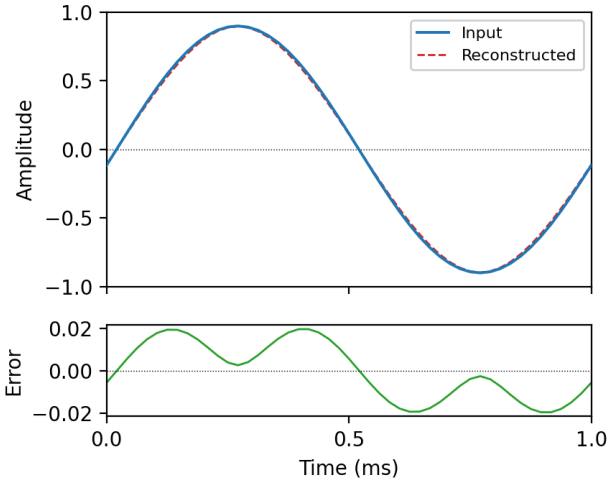

Figure 7: Time domain difference for a single cycle of sine.

Low amplitude noise is another artifact that manifests either when subsample extrema placement isn't used, or if the index becomes so large that the floating point numbers lose decimal precision. See Further Work for a discussion of a delta-based indexing approach.

3.4. CPU Performance

To quantify CPU performance we utilize the processor's cycle counter (DWT) and calculate the percent utilization from it according to the following formula.

$$\text{CPU\%} = \frac{\Delta\text{DWT\_CYCCNT}}{\left(\frac{f_{\text{CPU}}}{f_s}\right) \cdot N_{\text{chunk}}} \times 100 \tag{14}$$

We measured the CPU usage of a couple of effects from the DaisySP library in stereo as reference points. A 128-tap FIR filter averages 22% CPU, a pitch shifter effect averages 11% CPU, and a chorus effect averages 5% CPU. Results show that analysis is slightly more expensive than reconstruction, with the amount of compute required

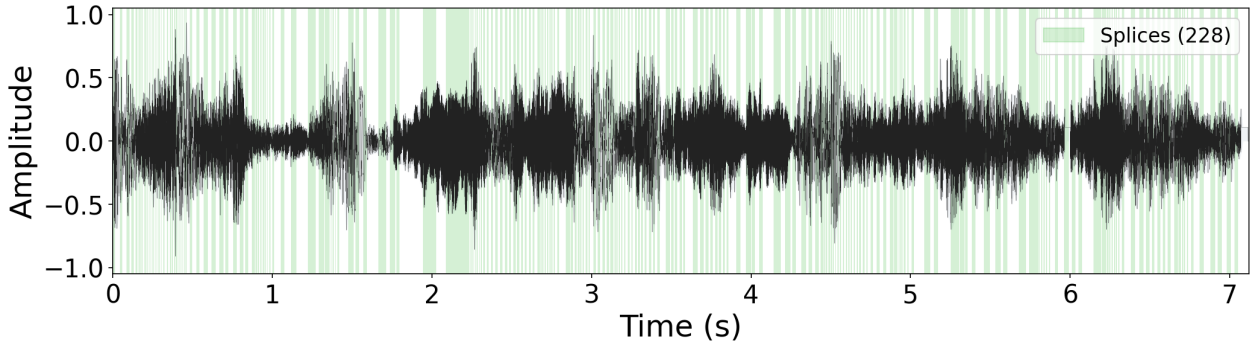

Figure 8: Demonstrating the adaptive splice duration on a musical sample with a 5.69:1 ratio, a pitch rate of 1.65, a time rate of 0.84, a grain size of 113 keyframes, and a total of 228 splices.

Table 2: CPU utilization during playback at the original sample rate (%, stereo L+R) averaged over 38 trials

	Analysis		Reconstruction	
Quality	Avg	Peak	Avg	Peak
High	6.22%	7.58%	4.89%	6.85%
Med	5.84%	7.22%	4.48%	6.50%
Low	5.63%	6.98%	4.31%	6.27%

for each depending on the information density of the signal. The peak numbers suggest that even densely spaced extrema aren't too taxing on the CPU. It's worth noting these percentages are relative to sample rate.

In a "record then play back" paradigm, analysis and reconstruction run separately. In the case of live feedback (see limitations point 3) they run simultaneously and their CPU usage should be summed. For time stretched playback, the CPU overhead relative to base reconstruction averages less than 1%, with a maximum delta of 1.76% at 2x speed. Extreme ratios of speed and pitch combined with small grain sizes increase CPU due to more frequent window searches, but the overall cost remains modest.

3.5. Perceptual Quality

To measure the apparent quality of the reconstruction at the original sample rate, we created a webMUSHRA test with three select audio clips and three select speech clips. For each clip, three levels of quality are tested alongside the input, an ADPCM filtered version, and a 3.5 kHz low pass filtered version. Mean scores (0–100) across 18 listeners are shown in Table 3. The results show that our high quality level is rated as fair and low quality is poor. User ratings scale appropriately with the threshold we chose for each quality level. While not exactly stellar in terms of raw numbers, this is expected as the method is inherently not transparent.

Table 3: webMUSHRA mean scores (18 listeners)

Trial	Ref	High	Med	Low	ADPCM	LP35
Music 1	93.8	55.5	31.9	14.7	92.6	37.8
Music 2	93.0	47.9	32.8	10.0	88.5	38.2
Music 3	91.6	64.1	27.9	18.4	92.9	40.9
Speech 1	94.1	49.2	30.1	18.4	86.0	51.1
Speech 2	88.9	62.9	34.9	22.8	85.4	47.1
Speech 3	88.8	50.4	31.3	19.9	86.7	52.1

User feedback from the survey indicates that although the artifacts in medium/low quality examples were prominent, their lo-fi character was desirable as a creative audio effect. Given how high the SNR values are for those quality levels, that says something about preservation of the signal within the noise.

3.6. Time Scale Modification Accuracy

To quantify the accuracy of our adaptive time stretching, we compute the MFCC cosine distance between the compressed but otherwise unmodified reference and each stretched output across 6 musical examples at 4 speed ratios and 2 grain sizes. Table 4 shows the results averaged over all trials.

Table 4: Time stretch accuracy and splice duration averaged over 6 trials. Splice duration shown as average.

	MFCC Cos. Dist.		Splice Duration (ms)	
Speed	$K=150$	$K=300$	$K=150$	$K=300$
0.50x	0.0825	0.1111	22.5	44.5
0.75x	0.0672	0.0941	22.8	46.8
1.50x	0.0375	0.0621	20.2	40.2
2.00x	0.0440	0.0903	21.6	44.8

In general, too small a grain size results in metallic AM sidebands, and too large results in audible repeats. Choosing an appropriate grain threshold K depends heavily on the contents of the sparse signal and the quality at which it was recorded. Lower quality recordings generally stretch more effectively with small grain sizes (16-32 keyframes), whereas for high quality recordings a range of 150-512 is most effective.

It's difficult to quantify, but empirically we found that when given real time control over grain size, users easily adjust it by ear to minimize both AM sidebands and audible repeats for a given time and pitch.

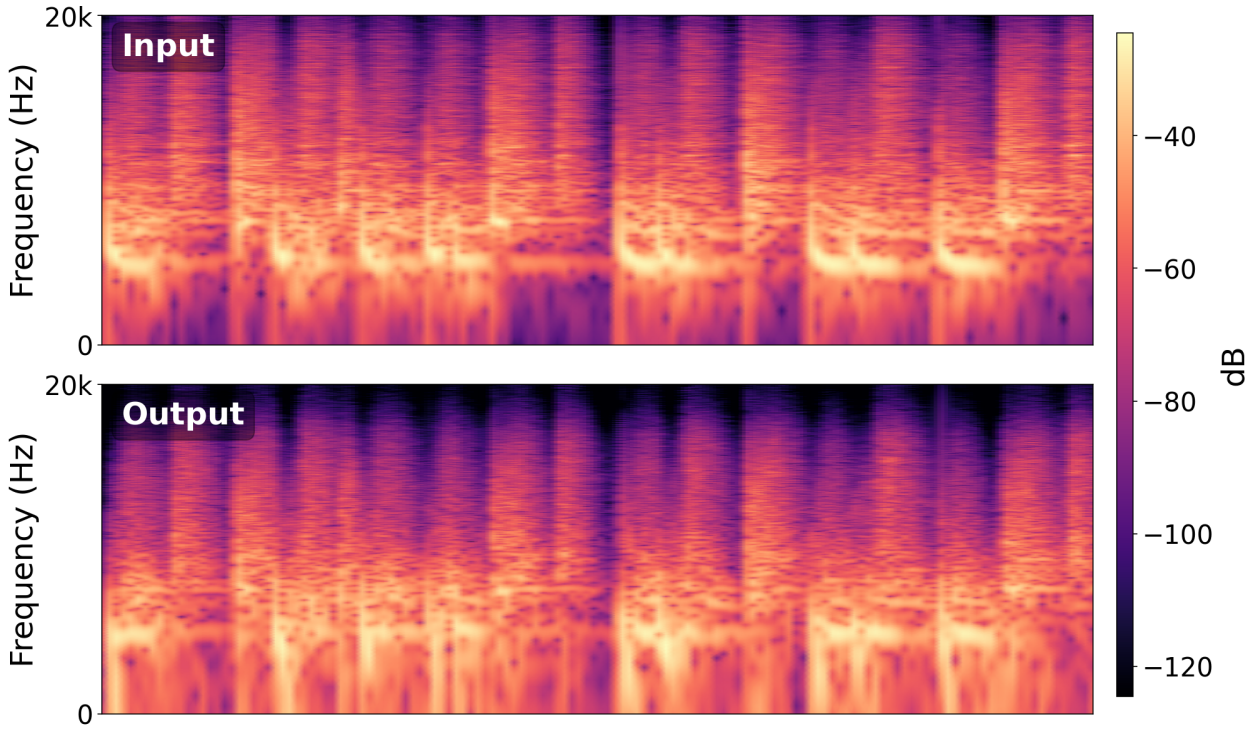

Figure 9: A drum loop with a 3.7:1 ratio time stretched to 2x speed at a grain size of 15 keyframes, with 756 total splices. Note the alignment of transients and the preservation of their highest frequencies (up to a point) and the boosting of sub bass frequencies.

3.7. Limitations

This approach is not without issues. We address what we consider the most important ones below:

1. Distortion dominates the reconstruction at low quality levels, despite the effective preservation of the signal which is still audible among the noise.
2. The maximum frequency this method can represent is determined by the analysis sample rate. Increasing the ADC's sample rate or oversampling digitally reduces the presence of aliasing.
3. This method requires future information to produce a reconstructed output. To enable real time processing we propose forcing keyframes at block boundaries. Block size sets the system latency and the maximum compression ratio.
4. This method is most effective when input volume utilizes the full dynamic range of the ADC. For a signal with a max amplitude far below the ADC's maximum, the effective bit depth of the operation is reduced, and different thresholds are needed.
5. Long periods of silence or otherwise sparse keyframes followed by transients result in a very long splice duration, meaning many audible repeating segments, potentially seconds long. To resolve this superficially we enforce an arbitrary maximum splice duration of a couple hundred milliseconds.

3.8. Further Work

Implementing extrema detection in the analog domain with an FPGA or custom silicon where a hardware differentiator triggers the ADC to save timestamped data would dramatically increase the timing accuracy, eliminate aliasing, and remove the analysis compute requirement entirely.

One potential strategy to resolve the loss of decimal precision that happens when 32 bit floating point numbers become large is a delta based indexing system in which only the Δ is saved instead of the full index. The full index is stored only at block boundaries which are already a necessity for real time monitoring.

Our 64 bit sparse format of two floats per point has the potential to be quantized to 16 bit integers. Although outside the scope of this paper, we believe that this reduction of precision in amplitude and continuous index would be outweighed by the storage benefits.

We experimented with making the quality threshold adaptive by calculating the Teager Kaiser energy operator (TKEO) from the analysis derivatives then passing the result through a low pass filter and using it to modulate the threshold during recording. This improves compression ratios without loss of quality when done right.

Another proposed extension to this method is a multi resolution analysis via spline wavelets to separate the input into frequency bands then run the keyframe analysis on each band. The prevalence of multi-rate filter banks in other compression algorithms bodes well for the success of such an approach.

4. CONCLUSION

This method's advantage is that the signal's information density is baked into the sparse data structure. That information can then be leveraged in operations like time stretching at virtually no added cost. The results show the high quality level to be generally acceptable, and at that level we get about a 3:1 compression ratio with the primary artifacts being some harmonic saturation and increased noise floor.

The cost of analysis? Two cubic interpolations per sample to detect the extrema, and another if an extremum is detected. Linear interpolation and a couple multiplies. The cost of reconstruction? Slightly less than the analysis. The cost of time stretching? Negligible even at extreme ratios. Sure, we lose random access to the sparse buffer, but localized searches keep cache demand to a minimum.

Our signal representation offers moderate compression and a framework for creative sound manipulation and degradation. The simple nature of the algorithm is well suited to low cost embedded microcontrollers. We view this not as a codec but as an alternative 1D signal representation that offers a unique perspective on time domain signals *of any kind* at a low computational cost.

5. REFERENCES

[1] K. Brandenburg, "MP3 and AAC explained," in *Proc. AES 17th Int. Conf. on High Quality Audio Coding*, Florence, Italy, Sept. 1999.

[2] N. E. Huang, Z. Shen, S. R. Long, M. C. Wu, H. H. Shih, Q. Zheng, N.-C. Yen, C. C. Tung, and H. H. Liu, "The empirical mode decomposition and the Hilbert spectrum for nonlinear and non-stationary time series analysis," *Proc. Royal Soc. London A*, vol. 454, pp. 903–995, 1998.

[3] N. Sayiner, H. V. Sorensen, and T. R. Viswanathan, "A level-crossing sampling scheme for A/D conversion," *IEEE Trans. Circuits Syst. II*, vol. 43, no. 4, pp. 335–339, Apr. 1996.

[4] G. Essl, "Topology-preserving deformations of digital audio," in *Proc. 27th Int. Conf. Digital Audio Effects (DAFx24)*, Guildford, UK, Sept. 3–7, 2024.

[5] E. Moulines and F. Charpentier, "Pitch-synchronous waveform processing techniques for text-to-speech synthesis using diphones," *Speech Communication*, vol. 9, no. 5–6, pp. 453–467, 1990.

[6] W. Verhelst and M. Roelands, "An overlap-add technique based on waveform similarity (WSOLA) for high quality time-scale modification of speech," *Proc. IEEE Int. Conf. Acoustics, Speech and Signal Processing (ICASSP)*, vol. 2, pp. 554–557, 1993.